WEEK 5 TASK

PAYMENT ENTITY – SQL DATABASE REPORT

1. Objective

To create and manage a Payment Entity using SQL by implementing table creation, constraints, payment records, and queries for successful, failed and pending transactions.

2. Payment Entity

The Payment Entity is used to store and manage payment information associated with customer orders.

Attributes

Attribute	Description
Payment_ID	Unique identifier for each payment
Order_ID	Identifies the order associated with the payment
Payment_Date	Date on which the payment is made
Payment_Mode	Method used for payment, such as UPI, Card, or Cash
Payment_Status	Current status of the payment: Success, Failed, or Pending

Transaction_Amount Total amount paid for the transaction

3. Create Payment Table

The Payment table is created to store payment-related information.

```
SQL> CREATE TABLE Payment (
  2       PAYMENT_ID VARCHAR2(12),
  3       ORDER_ID CHAR(7),
  4       PAYMENT_DATE DATE,
  5       PAYMENT_MODE VARCHAR2(15),
  6       PAYMENT_STATUS VARCHAR2(15),
  7       TRANSACTION_AMOUNT NUMBER(8,2)
  8  );

Table created.
```

4. Constraints

Constraints are applied to maintain data integrity and establish the relationship between the Payment and Orders tables.

Primary Key

```
Table created.

SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT PK_PAY PRIMARY KEY (PAYMENT_ID);

Table altered.
```

Foreign Key and NOT NULL Constraint

```
SQL> ALTER TABLE Payment
  2  MODIFY ORDER_ID NUMBER;

Table altered.

SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT FK_PAY_ORD
  3  FOREIGN KEY (ORDER_ID)
  4  REFERENCES Orders(ORDER_ID);

Table altered.

SQL> ALTER TABLE Payment
  2  MODIFY PAYMENT_DATE NOT NULL;

Table altered.
```

Check Constraint

```
SQL> ALTER TABLE Payment
  2  ADD CONSTRAINT CHK_PAY_AMT
  3  CHECK (TRANSACTION_AMOUNT > 0);

Table altered.
```

5. Insert Payment Records

Payment records are inserted into the Payment table with different payment modes, dates, statuses and transaction amounts.

```
SQL> INSERT INTO Payment VALUES ('PAY302', 1002, DATE '2026-10-16', 'Cash', 'Pending', 4500);

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES ('PAY303', 1003, DATE '2026-10-17', 'Debit Card', 'Success', 3400);

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES ('PAY304', 1004, DATE '2026-10-18', 'Credit Card', 'Success', 7500);

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES ('PAY305', 1005, DATE '2026-10-19', 'UPI', 'Failed', 1500);

1 row created.

SQL> SELECT * FROM Payment;
```

```
SQL> SELECT * FROM Payment;

PAYMENT_ID     ORDER_ID PAYMENT_D PAYMENT_MODE    PAYMENT_STATUS
---------- ---------- --------- --------------- ---------------
TRANSACTION_AMOUNT
------------------
PAY301             1001 15-OCT-26 UPI             Success
              2800

PAY302             1002 16-OCT-26 Cash            Pending
              4500

PAY303             1003 17-OCT-26 Debit Card      Success
              3400


PAYMENT_ID     ORDER_ID PAYMENT_D PAYMENT_MODE    PAYMENT_STATUS
---------- ---------- --------- --------------- ---------------
TRANSACTION_AMOUNT
------------------
PAY304             1004 18-OCT-26 Credit Card     Success
              7500

PAY305             1005 19-OCT-26 UPI             Failed
              1500
```

6. Display All Successful Payments

This query displays all payment transactions whose status is Success.

```
SQL> SELECT * FROM Payment
  2  WHERE PAYMENT_STATUS = 'Success';

PAYMENT_ID     ORDER_ID PAYMENT_D PAYMENT_MODE    PAYMENT_STATUS
---------- ---------- --------- --------------- ---------------
TRANSACTION_AMOUNT
------------------
PAY301             1001 15-OCT-26 UPI             Success
              2800

PAY303             1003 17-OCT-26 Debit Card      Success
              3400

PAY304             1004 18-OCT-26 Credit Card     Success
              7500
```

→ Successful Payments result

7. Find All Failed Transactions

This query displays all payment transactions whose status is Failed.

```
SQL> SELECT * FROM Payment
  2  WHERE PAYMENT_STATUS = 'Failed';

PAYMENT_ID     ORDER_ID PAYMENT_D PAYMENT_MODE    PAYMENT_STATUS
---------- ---------- --------- --------------- ---------------
TRANSACTION_AMOUNT
------------------
PAY305             1005 19-OCT-26 UPI             Failed
              1500
```

→ Failed Payments result

8. Count Total Successful and Failed Payments

This query counts the total number of successful and failed payments.

```
SQL> SELECT PAYMENT_STATUS, COUNT(*) AS TOTAL_PAYMENTS
  2  FROM Payment
  3  WHERE PAYMENT_STATUS IN ('Success', 'Failed')
  4  GROUP BY PAYMENT_STATUS;

PAYMENT_STATUS  TOTAL_PAYMENTS
--------------- --------------
Failed                       1
Success                      3
```

→ Count result

9. Update Failed Payments After Retry

This query updates failed payments to Success after a successful retry.

```
SQL> UPDATE Payment
  2  SET PAYMENT_STATUS = 'Success'
  3  WHERE PAYMENT_ID = 'PAY305'
  4  AND PAYMENT_STATUS = 'Failed';

0 rows updated.
```

→ UPDATE result

10. Identify Pending Transactions

This query displays all payment transactions whose status is Pending.

```
SQL> SELECT * FROM Payment
  2  WHERE PAYMENT_STATUS = 'Pending';

PAYMENT_ID     ORDER_ID PAYMENT_D PAYMENT_MODE    PAYMENT_STATUS
---------- ---------- --------- --------------- ---------------
TRANSACTION_AMOUNT
------------------
PAY302             1002 16-OCT-26 Cash            Pending
              4500
```

→ Pending Transactions result

11. Conclusion

The Payment Entity was successfully created and implemented using SQL. Primary key, foreign key, NOT NULL and check constraints were applied to maintain data integrity. Payment records were inserted and analyzed using SQL queries. Successful, failed and pending transactions were identified and managed according to their status.